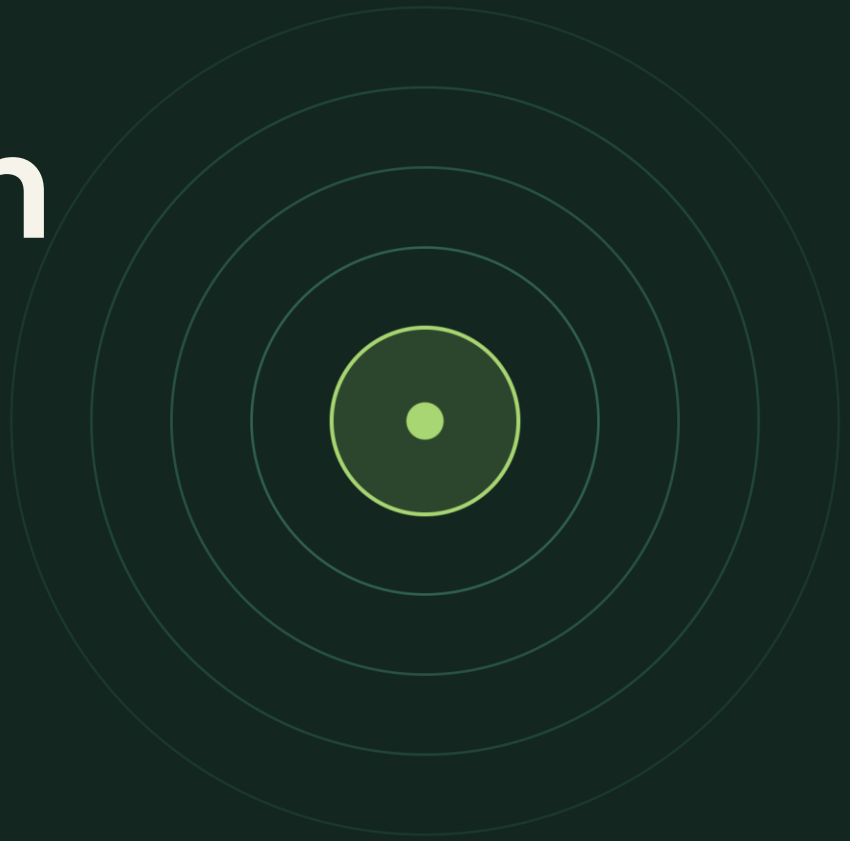
23CSE455 DESIGN PATTERNS · CASE STUDY

Design Patterns in Mycodo



TEAM B10 · AIE

Johan Rajesh
AM.SC.U4AIE23038

Aadarsh Jayan Nair
AM.SC.U4AIE23003

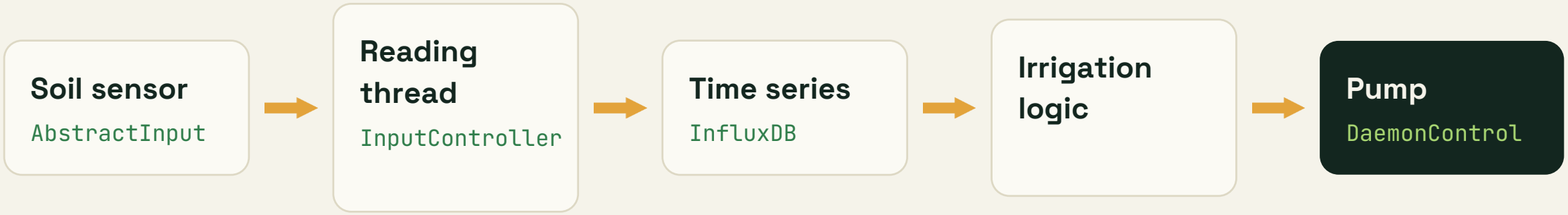
Dano Aby Alex
AM.SC.U4AIE23029

P Arya
AM.SC.U4AIE23140

THE APPLICATION

Mycodo: a farm controller in Python

An open-source monitoring and regulation system: a Flask web UI plus a daemon of controller threads. We use it as an agriculture manager that reads the soil, decides, and waters the crop.



PART 1 · PATTERN DISCOVERY

What is already inside Mycodo?

8
pattern types

13
confirmed instances

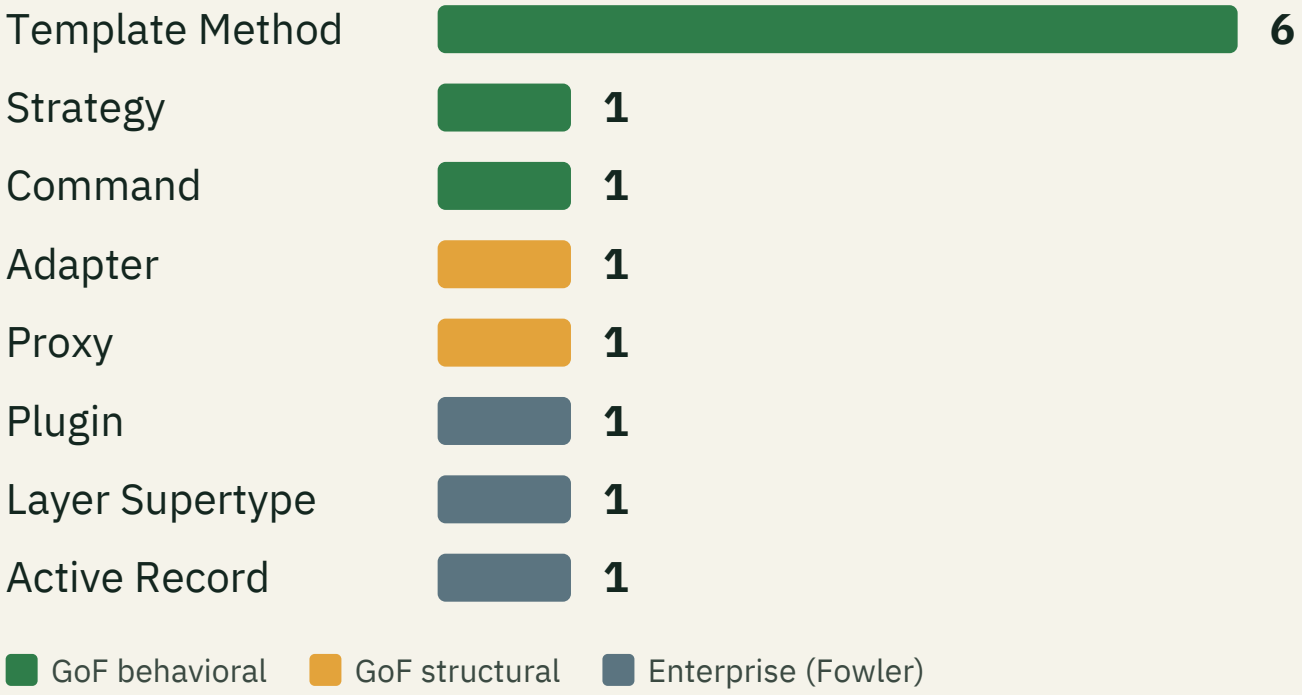
18
concrete classes

28
candidates rejected (23) or uncertain (5)

A pattern counted only when the collaboration was visible in the code: who calls whom, who extends whom, at which line. A class name alone was never enough.

PATTERN INVENTORY

Template Method carries the design



6 of 13

instances are Template Method: a base class fixes the algorithm and each device module fills in one step.

Plugin, Layer Supertype and Active Record shape the architecture around them.

HIGHLIGHT · TEMPLATE METHOD

One skeleton, many sensors

AbstractInput

`read()` — fixed skeleton: errors, retries, storing values

`get_measurement()` — the one step a sensor fills in

extends

extends

adafruit_i2c_soil

soil moisture over I²C

weather_openweathermap

city weather over HTTP

`inputs/base_input.py:115-146`

- ✓ The base class owns the algorithm; subclasses never call it in the wrong order.
- ✓ A new sensor is one subclass; the controller does not change.
- ✓ The same shape repeats for initialisation, thread loops, outputs and setpoints.

Swappable schedules, hidden network

STRATEGY

Setpoint Schedule Calculation

Pick the algorithm by type; callers only ask for the setpoint.

```
type = 'DailySine'
```

↓

```
create_method_handler()
```

↓

```
DailySineMethod.calculate_setpoint()
```

PROXY

Daemon RPC Client

Looks like the daemon; every call travels over Pyro5 RPC.

```
bang_bang.loop()
```

↓

```
DaemonControl.output_on()
```

↓

```
Pyro5 Proxy → daemon process
```

PART 2 · EVOLUTION AND REFACTORING

Three features a farmer would ask for

CH01

Crop types

Choose tomato, rice or wheat; the target moisture comes from the crop.

Duplicated Code → Extract Method

`utils/crop_profiles.py`

CH02

Irrigation strategies

Water only in time windows, skip when it rains, and stack the rules.

Switch Statements → Replace Conditional with Polymorphism

`utils/irrigation_strategies.py`

CH03

Sensor alerts

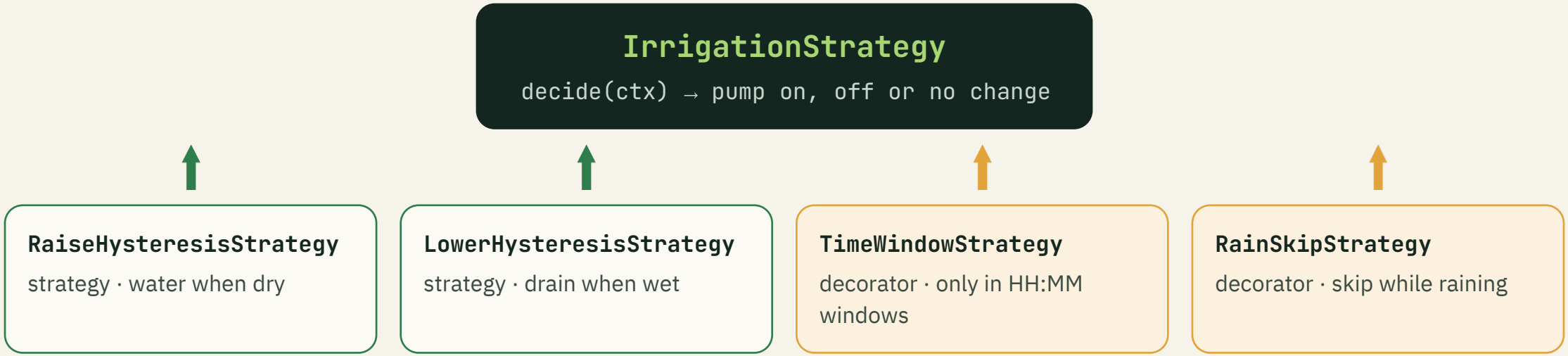
Warn when the soil is critically dry or the sensor goes silent.

Long Method → Extract Method

`utils/sensor_alerts.py`

CH02 · STRATEGY + DECORATOR

Irrigation policies become objects



Decorators wrap any strategy and stack

`RainSkip(TimeWindow(RaiseHysteresis()))`

loop() complexity after refactoring

CCN 14 → 7

CH03 · OBSERVER

Readings turn into alerts

`bang_bang.loop()`
detects a missing reading or critically dry soil

→

`SensorAlertNotifier`
subject: attach · detach · raise_alert · clear

→

`LogAlertListener`
always attached

`EmailAlertListener`
when recipients are set

Once per episode; recovery re-arms it

A failing listener never stops irrigation

Also fixes a crash on empty readings

EVOLUTION RESULTS

Snapshot	Types	Instances	Concrete classes	New patterns	Tests
Original code	8	13	18	none	25
After CH01	8	13	18	none	42
After CH02	9	15	22	Strategy, Decorator	64
After CH03	10	16	24	Observer	83

PART 3 · CROSS-LANGUAGE COMPARISON

One problem per pattern, four languages

8 patterns × **4** languages = **32** programs

32/32 test runners pass

Java · OpenJDK 21 | **Python** · 3.12 | **JavaScript** · Node 23 | **C++14** · g++ 6.3

Each language reads the same CSV of 9–12 cases, including error cases, so every implementation is judged on identical inputs and expected outputs.

STATIC MEASURES

Mean source lines per production file (8 patterns)



Highest function complexity per file stays close: Python 6–19 · JavaScript 5–22 · Java 7–23 · C++ 7–25.

BIGGEST GAP: PROXY

Python	Java
60	111

__getattr__ and the JavaScript Proxy object forward any call; Java and C++ must declare and repeat every method of the interface.

LLM USE AND VERIFICATION

LLMs drafted, we verified

- Each member ran all three parts with their own prompt
- Prompts, inputs and raw outputs kept in every llm/ folder
- Every claim graded in llm.csv: correct, partial or cannot confirm

112/112

Part 1 line ranges checked by script

115 tests

83 Part 2 + 32 Part 3, re-run after every change

Rebuilt

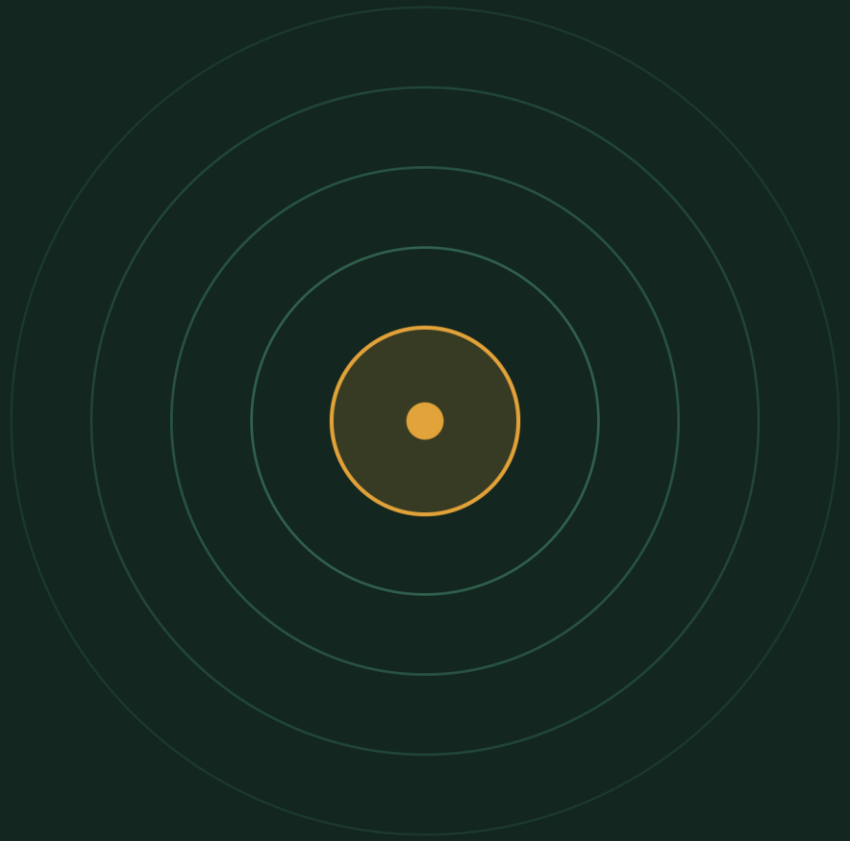
result CSVs regenerated from raw tool output

Fixed

mislabelled totals, a lizard pass that skipped .hpp files

Thank you

Questions?



Report, data and code

`github.com/aswathymohan-amrita/23CSE455_DP_CaseStudy · B10/`

Johan Rajesh · Aadarsh Jayan Nair · Dano Aby Alex · P Arya